

Oriented Bounding Boxes Object Detection

Oriented Bounding Boxes Object Detection

- 1、 Model Introduction
- 2、 Oriented Bounding Boxes Object Detection: Image
 - Effect Preview
- 3、 Oriented Bounding Boxes Object Detection: Video
 - Effect Preview
- 4、 Oriented Bounding Boxes Object Detection: Real-time Detection
 - 4.1、 USB Camera
 - Effect Preview
 - Reference Materials

This article demonstrates **Ultralytics** Oriented Bounding Boxes Object Detection on images, videos, and real-time detection using Python.

1、 Model Introduction

Oriented bounding boxes object detection, also known as oriented object detection, goes a step further than standard object detection by introducing an additional angle parameter to locate objects in images more accurately.

The output of an oriented object detector is a set of rotated bounding boxes that precisely enclose the objects in the image, along with the class label and confidence score for each bounding box. Oriented bounding boxes are particularly useful when objects appear at different angles, such as in aerial imagery, where traditional axis-aligned bounding boxes may include unnecessary background areas.

In simple terms, oriented object detection can use **tilted boxes** to precisely enclose **tilted objects**, thereby reducing unnecessary background areas and improving detection accuracy.

2、 Oriented Bounding Boxes Object Detection: Image

Use yolo26n-obb_ncnn_model to predict images in the ultralytics26 folder (non-ultralytics built-in images).

Enter the code folder:

```
cd /home/sunrise/ultralytics26/yahboom_demo
```

Run the code:

```
python3 05.obb_image.py
```

Effect Preview

YOLO recognized output image location: /home/sunrise/ultralytics26/output/



Example code:

```
from ultralytics import YOLO

# Load a model
model = YOLO("/home/sunrise/ultralytics26/yolo26n-obb_ncnn_model")
# model = YOLO("/home/sunrise/ultralytics26/yolo26n-obb.pt")

# Run batched inference on a list of images
results = model("/home/sunrise/ultralytics26/assets/car.jpg") # return a list of Results
objects

# Process results list
for result in results:
    boxes = result.boxes # Boxes object for bounding box outputs
    # masks = result.masks # Masks object for segmentation masks outputs
    # keypoints = result.keypoints # Keypoints object for pose outputs
    # probs = result.probs # Probs object for classification outputs
    obb = result.obb # Oriented boxes object for OBB outputs
    result.show() # display to screen
    result.save(filename="/home/sunrise/ultralytics26/output/car_output.jpg") # save to
disk
```

3、Oriented Bounding Boxes Object Detection: Video

Use yolo26n-obb_ncnn_model to predict videos in the ultralytics26 folder (non-ultralytics built-in videos).

Enter the code folder:

```
cd /home/sunrise/ultralytics26/yahboom_demo
```

Run the code:

```
python3 05.obb_video.py
```

Effect Preview

YOLO recognized output video location: /home/sunrise/ultralytics26/output/



Example code:

```
import cv2
from ultralytics import YOLO

# Load the YOLO model
model = YOLO("/home/sunrise/ultralytics26/yolo26n-obb_ncnn_model")
# model = YOLO("/home/sunrise/ultralytics26/yolo26n-obb.pt")

# Open the video file
video_path = "/home/sunrise/ultralytics26/videos/street.mp4"
cap = cv2.VideoCapture(video_path)
```

```

# Get the video frame size and frame rate
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))

# Define the codec and create a Videowriter object to output the processed video
output_path = "/home/sunrise/ultralytics26/output/05.street_output.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v') # You can use 'XVID' or 'mp4v' depending on your platform
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO inference on the frame
        results = model(frame)

        # Visualize the results on the frame
        annotated_frame = results[0].plot()

        # Write the annotated frame to the output video file
        out.write(annotated_frame)

        # Display the annotated frame
        cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

        # Break the loop if 'q' is pressed
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    else:
        # Break the loop if the end of the video is reached
        break

# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()

```

4、Oriented Bounding Boxes Object Detection: Real-time Detection

4.1、USB Camera

Use yolo26n-obb_ncnn_model to predict USB camera feed.

Enter the code folder:

```
cd /home/sunrise/ultralytics26/yahboom_demo
```

Run the code: Click the preview window and press the q key to terminate the program!

```
python3 05.obb_camera_usb.py
```

Effect Preview

YOLO recognized output video location: /home/sunrise/ultralytics26/output/



Example code:

```
import cv2
from ultralytics import YOLO

# Load the YOLO model
model = YOLO("/home/sunrise/ultralytics26/yolo26n-obb_ncnn_model")
# model = YOLO("/home/sunrise/ultralytics26/yolo26n-obb.pt")

# Open the camera
cap = cv2.VideoCapture(0)
cap.set(6, cv2.VideoWriter_fourcc('M', 'J', 'P', 'G'))
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)

# Get the video frame size and frame rate
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
```

```

frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))

# Define the codec and create a VideoWriter object to output the processed video
output_path = "/home/sunrise/ultralytics26/output/05.obb_camera_usb.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v') # You can use 'XVID' or 'mp4v' depending on your
platform
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO inference on the frame
        results = model(frame)

        # Visualize the results on the frame
        annotated_frame = results[0].plot()

        # Write the annotated frame to the output video file
        out.write(annotated_frame)

        # Display the annotated frame
        cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

        # Break the loop if 'q' is pressed
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    else:
        # Break the loop if the end of the video is reached
        break

# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()

```

Reference Materials

[Oriented Bounding Boxes Object Detection - Ultralytics YOLO Docs](#)